

PortFlow Upload Guide -- Marketing Agent POC

COLLECTION DETAILS

Collection Title

Marketing Agent -- AI-Powered Campaign Studio

Collection Description (paste this into PortFlow)

What is this project about? Marketing Agent is a multi-agent AI system that takes a product image and short description and produces a complete marketing campaign in under 90 seconds. Eight specialised agents collaborate: one analyses the product image, one segments the target audience, one writes two ad copy variants with distinct angles, one checks for compliance violations, one scores and improves the CTAs. A RAG layer lets users inject brand briefs or product specs. A live refinement panel lets them iterate on tone, audience age, and copy without re-running the full pipeline.

Why does it exist? The project responds to a documented shift in the industry: one Dutch advertising agency went from 290 copywriters to 20 after implementing AI. The assignment asked the same question Reclamebureau Eva answered on Dutch national television: can a team of specialised AI agents produce professional campaign output that a single generalist agent cannot? The system was built to test that question with real evidence, not theory.

What was actually accomplished? A fully working end-to-end system: a Flask backend with SSE streaming progress, a two-column result UI with a sticky refinement panel, eight specialised agents with clean input/output contracts, a ChromaDB RAG layer, ReAct routing that re-runs only the affected pipeline stages on each refinement, and an append-only telemetry log that tracks output quality across runs. Every architectural decision is documented in a nine-entry Decision Log that includes the research question, the LO stage progression, evidence, and what the decision unlocked.

PHASE STORY (Phase 0 -> Phase 3)

Based on decision log arc and development sequence.

Phase 0 -- Analyzing: what does an agency actually do?

Before writing any agent code, I worked through six questions about agency function using two research sources. The finding that mattered: even the smallest agency performs exactly five distinct functions -- product analysis, audience definition, copy generation, CTA optimisation, compliance review. Those five became the pipeline. Ten functional requirements were written, five scoped for the POC, five explicitly deferred. Compliance made the required set because it's a legal requirement, not polish.

Key changes: - feat: scaffold project with base five-agent pipeline -- vision, audience, copywriter, guardrails, CTA - docs: add agency research and requirements spec -- five-function baseline, FR1-FR10, five deferred items documented

DLs produced: A1 (agency research), A2 (requirements spec), DL-00 (scope + model stack)
What it unlocked: A fixed pipeline target. Five agents, no scope drift.

Phase 1 -- Designing -> Realizing: monolith first, then specialists

The monolith was built first to have a real comparison. One Ollama call, one prompt: output on the eco water bottle was "Check out this amazing product!" with audience "general public." The multi-agent pipeline on the same image, same model: "Eco-Conscious Millennial Athlete, 25-34." Same model. The difference was each agent receiving a narrow scope and only the context it needed.

Two bugs surfaced immediately. The vision model (gemma-3-27b-it:free) returned HTTP 400 -- text-only, no vision support despite the name. Switched to nemotron + llama fallback and redesigned the output schema from visual descriptions to target_signals. For compliance, an LLM judge was tested against five planted violations and missed two -- decided a guarantee claim was acceptable in context. That's a legal call it can't make. Switched to pure Python regex: seven patterns, under 1ms, zero false negatives.

Key changes: - feat: build monolith baseline then decompose -- audience agent added between vision and copywriter - fix: switch vision model to nemotron + llama fallback after gemma HTTP 400; redesign output schema to target_signals - feat: add guardrails agent -- pure Python regex after LLM judge missed 2/5 violations in testing

DLs produced: DL-01 (monolith -> specialists), DL-02 (vision agent), DL-03 (guardrails)

What it unlocked: A working pipeline with observable proof that specialisation improves output.

Phase 2 -- Realizing: RAG layer and interactive refinement

Campaigns were generic when the product had specs not visible in the image. Added ChromaDB + nomic-embed-text -- both local, both pip install, no infrastructure. Copy generated with a brand brief uploaded referenced specific product claims. Copy without it was generic.

The one-shot report wasn't a usable workflow. Designed the two-column layout in wireframe.txt before writing any HTML -- sticky panel, sliders above chat, routing label visible. ReAct routing uses one LLM call to decide which agents re-run; vision never re-runs during refinement. SSE streaming replaced the blank-screen wait. Also: the copywriter timed out at 60 seconds on a long-context run -- mistral:7b stalled on creative prose. Split the config: gemma4:e2b for copywriter, mistral:7b for all JSON tasks.

Key changes: - feat: add RAG pipeline -- ChromaDB + nomic-embed-text, local, persistent, deduplicated - feat: add ReAct assistant with SSE streaming -- one routing call, stateless round-trips, live progress overlay - fix: split model config -- gemma4:e2b for copywriter, mistral:7b for JSON tasks after 60s timeout

DLs produced: DL-04 (RAG), DL-05 (refinement UI + routing), DL-06 (model split)

What it unlocked: An interactive tool. Generate -> adjust -> refine. 90-second generation, 30-second refinement cycle.

Phase 3 -- Managing: telemetry, validation, and the honest accounting

With the pipeline stable, the question shifted to what was actually happening across runs. `run_log.jsonl` records one line per run -- pipeline time, model used, compliance flag count, CTA score, persona label. First finding: compliance flags in ~40% of runs, almost all urgency language the user never asked for. The telemetry made it visible.

Acceptance tests, a one-command setup script, and a pre-demo health check were added. The ethics DL, required by the brief, named three accountability gaps: no inter-agent critique (errors cascade undetected), compliance coverage not exhaustive (novel phrasing slips through), and persona bias (model defaults, not market research). Documented so anyone using the output knows to apply human judgment before publishing.

Key changes: - feat: add `run_log.jsonl` telemetry and `/stats` endpoint -- metrics per run, first findings visible - feat: add `tests/validate.py`, `setup.py`, `health_check.py` -- acceptance tests and pre-demo verification - docs: add DL-07 (telemetry) and DL-08 (ethics -- three accountability gaps named)

DLs produced: DL-07 (pipeline telemetry), DL-08 (ethics)

What it unlocked: Evidence the system works -- and honest documentation of where it doesn't.

EVIDENCE FILE LIST WITH LO TAGS

All paths relative to the `marketingAI/` project root.

Decision Logs (primary evidence -- tag to multiple LOs)

File	Description	LO Tags
decision_logs/A1_agency_function_research.txt	Agency role research: six questions answered, five-fu	
decision_logs/A2_requirements_and_scope.txt	FR/BR/US spec with POC scope decisions and deferral rea	
decision_logs/ADV1_agent_decomposition.txt	Pre-build advisory: why five specialists, pipeline order	
decision_logs/ADV2_compliance_strategy.txt	Pre-build advisory: LLM judge vs regex -- risk profile,	
decision_logs/00_scope_and_model_stack.txt	Scope locked to five agents; cloud/local model split dec	
decision_logs/01_monolith_to_specialists.txt	Monolith comparison, specialisation proof, audience st	
decision_logs/02_vision_agent.txt	Model fallback chain + output schema redesign from visual -> k	
decision_logs/03_guardrails_pure_python.txt	LLM judge rejected; regex chosen; zero false negatives	
decision_logs/04_rag_chromadb.txt	ChromaDB + nomic-embed-text; copy improves with brand documents	
decision_logs/05_refinement_ui_routing.txt	Wireframe -> ReAct routing -> SSE progress; CTA ke	
decision_logs/06_model_split.txt	Timeout -> gemma4:e2b for copy, mistral:7b for JSON; one config	
decision_logs/07_pipeline_telemetry.txt	run_log.jsonl, /stats endpoint, first findings (40% complia	

decision_logs/08_ethics.txt		Workforce displacement, three accountability gaps, system limits		L0
decision_logs/STORY.md		10-minute narrative arc connecting all phases		L01, L02, L03, L04, L05
decision_logs/INDEX.txt		Reading guide and file registry		L01

Research and Analysis (LO1 -- Analyzing)

File		Description		L0 Tags
Project_case.md		Assignment brief: Reclamebureau Eva case, success criteria, ethics requirement		
2518cd57-..._AI_marketing_agent2.pdf		Agency function research source (ironpaper.com + setup.us)		
2ec2ea0f-..._Requirements3.pdf		Formal requirements spec: FR1-FR10, BR1-BR4, US-1 to US-5		L01
analysis/research_questions.md		Five formal research questions driving the project		L01
analysis/stakeholder_analysis.md		Four stakeholder groups: small business owner, student, displaced		
analysis/constraints.md		Technical, functional, design, scope, and educational constraints		L01
analysis/competitor_scan.md		Jasper, Copy.ai, ChatGPT scan -- why existing tools don't answer the re		

Design Documents (LO3 -- Designing)

File		Description		L0 Tags
wireframe.txt		ASCII wireframe of result.html two-column layout; 8 design decisions with rationale		
ARCHITECTURE.md		Agent pipeline diagram: vision -> color -> RAG -> audience -> copywrite		
config.example.py		Model stack configuration; OLLAMA_MODEL / OLLAMA_FAST_MODEL split		L03
docs/agent_contracts.md		Input/output contracts per agent -- what each receives, what each returns,		
docs/user_requirements.md		User needs from the upload page, result page, refinement panel; trust re		

Implementation (LO4 -- Realizing)

File		Description		L0 Tags
app.py		Flask orchestrator: SSE /generate, /refine, /result/, /export, /health, /stats		L04
agents/vision.py		Product analyst agent; nemotron primary, llama fallback; target_signals output		
agents/audience.py		Persona segmentation from product signals; mistral:7b JSON		L04

agents/copywriter.py	Two variants with enforced angles (emotional hook / product benefit); gemma4:0
agents/guardrails.py	Pure Python regex; seven compliance patterns; no LLM call L04
agents/cta_optimizer.py	CTA scoring and improvement suggestions per variant; mistral:7b L04
agents/rag.py	ChromaDB retrieval; nomic-embed-text embeddings; chunk deduplication L04
agents/assistant.py	ReAct routing, slider mapping, proactive suggestions; mistral:7b L04
agents/color.py	Dominant color extraction from product image L04
templates/index.html	Upload form with SSE progress overlay and step indicators L04
templates/result.html	Two-column report + sticky refinement panel + export button L04
templates/docs.html	Knowledge base management UI L04
tests/validate.py	Pipeline acceptance tests: compliance detection, persona specificity, char limits
scripts/setup.py	Install deps + pull Ollama models in one command L04
scripts/health_check.py	Pre-demo check: Ollama reachable, models present, OpenRouter key valid
requirements.txt	Python dependencies L04
README.md	Setup guide and project overview L04

Monitoring and Managing (LO5 -- Managing)

File	Description	LO Tags
app.py	/health + /stats routes	Health endpoint (Ollama status) and stats endpoint (run aggregates)
decision_logs/07_pipeline_telemetry.txt	run_log.jsonl design, /stats endpoint, first findings from	
decision_logs/08_ethics.txt	Three accountability gaps; system limits documented; workforce context	

QUICK LO SUMMARY

LO	What it covers in this project	Primary files
L01 Analyzing	Agency research, requirements spec, research questions, stakeholder analysis, constr	
L02 Advising	ADV1 (agent decomposition), ADV2 (compliance strategy), monolith comparison evidence,	
L03 Designing	Pipeline design, model stack, wireframe, agent contracts, user requirements, complian	
L04 Realizing	All agent implementation, Flask server, SSE streaming, refinement loop, tests, script	

UPLOAD ORDER RECOMMENDATION

Upload in this order so evidence is in place before DLs that reference it:

- 1 Collection description -- create the collection first
- 2 Sprint story (STORY.md) -- narrative anchor for the whole collection
- 3 Project_case.md + botqh PDFs -- establish the research context and brief
- 4 A1 + A2 research files -- LO1 foundation before any design files
- 5 analysis/ files (research_questions, stakeholder_analysis, constraints, competitor_scan)
-- supporting LO1 context
- 6 ADV1 + ADV2 advisory files -- LO2 pre-build recommendations before code
- 7 DL-00 (scope + model stack)
- 8 wireframe.txt + ARCHITECTURE.md + docs/ (agent_contracts, user_requirements) --
design artifacts together
- 9 DL-01 (monolith -> specialists) -- the key decision
- 10 DL-02 (vision agent) + DL-03 (guardrails)
- 11 Agent code files (agents/*.py) -- implementation evidence
- 12 app.py + templates/ -- Flask server and UI
- 13 DL-04 (RAG) + DL-05 (refinement UI)
- 14 DL-06 (model split)
- 15 tests/validate.py + scripts/ -- validation and tooling
- 16 DL-07 (telemetry) + DL-08 (ethics)
- 17 INDEX.txt + README last -- ties everything together